

SquareOps Technologies

DevOps Engineer – Interview Q&A Reference

Kubernetes • AWS • CI/CD • EKS • Networking

■ Kubernetes (Core + Advanced)

■ AWS Load Balancers

■ EKS Deployment

■ EC2 Access Recovery

■ CI/CD Pipeline

Prepared by: **Sanket Chopade** | DevOps Engineer

Company: SquareOps Technologies | Role: DevOps Engineer

SECTION 1 — Kubernetes: Core & Advanced

Q1. What is the current Kubernetes version you are working with?

The current stable Kubernetes version I work with is **v1.29.x** (as used in my ApnaKart EKS cluster). As of mid-2025, EKS supports up to **v1.30**.

- Always check: `kubectl version --short` to confirm client and server versions.
- EKS currently supports K8s 1.27 through 1.30; 1.30 is the latest GA on AWS.

■ *In interviews, cite the version tied to your project, not just the upstream latest. Interviewers check if you actually know what you ran.*

Q2. How do you perform a Kubernetes cluster upgrade?

A cluster upgrade follows a strict order: **control plane first, then worker nodes**. Skipping minor versions is not supported — always upgrade one minor version at a time.

End-to-End Upgrade Steps (EKS):

- **1. Review release notes:** Check deprecations and API changes for the target version.
- **2. Upgrade the Control Plane:** In EKS: update cluster version via Console, CLI, or Terraform. AWS manages etcd and API server.
- **3. Upgrade Managed Node Groups:** Rolling update via `eksctl` or Terraform — new nodes join, old ones drain automatically.
- **4. Upgrade Self-managed Nodes:** Cordon → Drain → Terminate old node → Launch new AMI node.
- **5. Upgrade Add-ons:** CoreDNS, kube-proxy, VPC CNI — each has its own compatible version per K8s release.
- **6. Update kubectl:** Client version should match or be within ± 1 minor of server version.
- **7. Validate:** Run smoke tests, check pod health, verify all daemonsets and deployments are Running.

■ *Never skip minor versions. 1.27 → 1.29 directly will break etcd and API compatibility.*

Q3. How does Kubernetes Ingress route traffic to services and pods?

Ingress is an API object that defines HTTP/HTTPS routing rules. It does **not** process traffic itself — it requires an **Ingress Controller** (e.g., NGINX, AWS ALB Controller) to implement the rules.

Traffic flow:

- Client → DNS → Load Balancer (NLB/ALB) → Ingress Controller Pod → Service (ClusterIP) → Pod (via kube-proxy / iptables / eBPF)

How pod routing stays accurate despite pod ephemerality:

- Pods register with a **Service**. The Service uses a **label selector** to maintain an **Endpoints** object.
- kube-proxy watches the Endpoints object and keeps iptables/IPVS rules updated in real time.
- When a pod dies, the Endpoints controller removes it. kube-proxy updates routing within seconds.
- Ingress routes to the Service name (stable DNS), not directly to pod IPs — so pod churn is abstracted away.

■ *The key insight: Ingress → Service (stable) → Endpoints (dynamic). Pods are ephemeral; the Service DNS is permanent.*

Q4. How does a pod get created when one replica is in a failed/pending state?

This is about the **reconciliation loop** of the **ReplicaSet Controller** (part of kube-controller-manager).

End-to-end flow:

- **kubelet** detects the pod has failed or is not heartbeating. It updates the pod's status in etcd.
- **ReplicaSet Controller** (in kube-controller-manager) watches pod state. It observes actual replicas (2) < desired replicas (3).
- Controller creates a new Pod object in the API server (etcd). Pod status: *Pending*.
- **kube-scheduler** detects the unscheduled pod and runs scheduling logic: node affinity, resource fit, taints/tolerations.
- Scheduler assigns the pod to a node by updating the pod spec's nodeName field.
- **kubelet** on the selected node watches for pods assigned to it. It pulls the image and starts containers via the CRI (containerd).
- Pod transitions: Pending → ContainerCreating → Running.

■ *The key components: kubelet (detects failure), controller-manager (creates replacement), scheduler (places it), kubelet again (runs it).*

Q5. How do you configure Horizontal Pod Autoscaler (HPA)?

HPA automatically scales the number of pod replicas based on observed metrics (CPU, memory, or custom metrics).

Prerequisites:

- Metrics Server must be installed in the cluster (kubectl top pods must work).
- The Deployment/StatefulSet must have **resource requests** defined — HPA uses these as the baseline for percentage calculations.

Basic HPA manifest:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
```

```
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70
```

- HPA checks metrics every **15 seconds** (default sync period).
 - Scale-up is aggressive; scale-down has a 5-minute cool-down by default to avoid thrashing.
- *Always set resource requests. HPA without requests = broken autoscaling.*

Q6. What is NLB (Network Load Balancer), and when would you use it?

NLB (Network Load Balancer) operates at **Layer 4 (TCP/UDP)**. It routes traffic based on IP and port — no HTTP header inspection.

Key characteristics:

- Extremely low latency (~microseconds) and can handle millions of requests per second.
- Preserves the client source IP address (unlike ALB, which terminates the connection).
- Supports static Elastic IPs — useful when clients whitelist IPs (e.g., partner firewalls).
- Supports TLS passthrough — encrypted traffic passes to the backend without NLB decrypting it.

When to use NLB over ALB:

- Non-HTTP workloads: gRPC, MQTT, custom TCP protocols, game servers, financial trading systems.
- When you need a static/fixed IP (NLB supports Elastic IPs; ALB does not).
- Ultra-low latency is a hard requirement.
- TLS passthrough is required (end-to-end encryption).

■ *In the SquareOps context: NLB typically fronts the Ingress Controller. ALB handles HTTP-layer routing.*

Q7. How do you expose Kubernetes services externally?

There are four main approaches, each with different trade-offs:

Method	Layer	Best For	Limitation
NodePort	L4	Dev/testing	Exposes high port (30000–32767), not production-grade
LoadBalancer	L4	Single service exposure	One cloud LB per service — expensive at scale
Ingress (+ Controller)	L7	HTTP/S, path & host routing, TLS termination	Needs Ingress Controller deployed
ExternalDNS + Route53	DNS	Auto DNS management	Companion tool, not standalone

■ *Production pattern on EKS: NLB → Ingress Controller (NGINX/ALB) → ClusterIP Services → Pods.*

Q8. In a Deployment with 3 replicas, one pod is in CrashLoopBackOff. Explain the end-to-end detection and replacement flow including kubelet, controller manager, and scheduler.

This tests deep understanding of the Kubernetes control plane. Here is the full flow:

Phase 1 – Detection (kubelet):

- The **kubelet** on the node runs liveness probes and monitors container exit codes.
- On repeated container crashes, kubelet applies exponential back-off before restarting (10s → 20s → 40s... up to 5 min). This is the **CrashLoopBackOff** state.

- kubelet updates the pod's status (phase: Running, condition: ContainersReady: False) back to the API server / etcd.

Phase 2 – Decision (kube-controller-manager):

- The **ReplicaSet controller** watches pod statuses continuously via the API server watch mechanism.
- It evaluates: *desired replicas* = 3, *healthy running* = 2. It does NOT count the CrashLoopBackOff pod as healthy.
- Controller creates a new Pod object (status: Pending) and writes it to etcd via the API server.

Phase 3 – Scheduling (kube-scheduler):

- kube-scheduler watches for unbound pods (pods with no nodeName set).
- Runs **filtering** (removes nodes that can't run the pod) and **scoring** (ranks remaining nodes).
- Binds the pod to the highest-scoring node by patching the pod's nodeName field in etcd.

Phase 4 – Execution (kubelet on selected node):

- kubelet on the target node sees the pod assigned to it. Pulls image (or uses cache), starts container via containerd.
- Pod transitions: Pending → ContainerCreating → Running.

What happens to the crashing pod?

- By default, the old crashing pod is NOT evicted immediately — Kubernetes continues restarting it.
- If the pod is stuck in CrashLoopBackOff with a long back-off, the **new replacement pod** runs alongside it.
- The ReplicaSet count goes to 4 temporarily (3 desired + 1 crashing). The controller will eventually evict the old pod once it determines it's failed (based on restart policy and status).

■ *Key insight: CrashLoopBackOff ≠ automatic eviction. The controller still counts the crashing pod. If the pod never reaches a terminal failed state, the RS may briefly overprovision.*

SECTION 2 — Ingress Deep Dive

Q9. How does Ingress redirect traffic to target pods? (Follow-up: Given pods are ephemeral, how is traffic routed accurately?)

Ingress itself is just a **configuration object** — it defines rules like 'route /api to service my-api on port 8080'. The actual traffic handling is done by the **Ingress Controller** (e.g., NGINX pod).

Full traffic path:

- External Client hits DNS → resolves to Load Balancer IP (NLB/ALB).
- LB forwards to Ingress Controller Pod (NGINX/Traefik/ALB Controller).
- Ingress Controller reads Ingress rules and proxies to the matching **Service (ClusterIP)**.
- Service has a stable DNS name and a virtual IP (ClusterIP). kube-proxy maintains iptables rules that forward from ClusterIP → actual Pod IPs.

Why pods being ephemeral isn't a problem:

- The **Endpoints controller** continuously watches pods matching the Service's label selector.
- When a pod dies: Endpoints object is updated (pod IP removed) → kube-proxy refreshes iptables → traffic stops going to dead pod — within seconds.
- When a new pod starts: pod IP is added to Endpoints → kube-proxy adds new iptables rule → new pod receives traffic.
- Ingress never routes directly to pod IPs. It always routes to the **Service**, which abstracts pod churn.

■ *Service = stable abstraction. Endpoints = dynamic pod IP list. Ingress routes to Service, not pods.*

Q10. Which problem does Ingress solve and how?

Before Ingress, exposing multiple services externally required one **LoadBalancer Service per application** — meaning one cloud LB (and its cost) per service. For 10 services = 10 LBs.

Problems Ingress solves:

- **Cost:** One external LB (via Ingress Controller) instead of one per service.
- **Host-based routing:** api.example.com → api-service, app.example.com → frontend-service — all on one IP.
- **Path-based routing:** /api → api-service, /static → cdn-service — on the same domain.
- **TLS termination:** Manage one SSL cert at the Ingress level instead of per-service.
- **Centralized access control:** Auth, rate-limiting, WAF rules applied at one choke point.

■ *One Ingress Controller = one cloud LB. All routing rules live as Ingress objects — cheap to create and version-controllable.*

Q11. If Ingress is already there, why do you still need NLB?

This is a common misconception. Ingress handles **Layer 7 (HTTP/S) routing inside the cluster**. But Ingress itself needs to be exposed to the outside world — that requires an external entry point, which is the **NLB (or ALB)**.

Why NLB specifically:

- The Ingress Controller (e.g., NGINX) runs as a pod. It needs a **Service of type LoadBalancer** to get an external IP — AWS provisions an NLB for this.
- NLB provides the fixed external IP/DNS that DNS records (Route53) point to.
- NLB handles raw TCP at Layer 4 — it doesn't inspect HTTP. The Ingress Controller handles all L7 routing after the NLB forwards TCP to it.
- NLB also handles **TLS passthrough** scenarios and **static IPs** that ALB cannot provide.

Architecture summary:

- Internet → NLB (L4, external entry) → Ingress Controller Pod (L7, HTTP routing) → ClusterIP Service → Pod

■ *Analogy: NLB is the front door of the building. Ingress is the directory inside that tells you which floor to go to.*

SECTION 3 — AWS Load Balancers

Q12. What are the different types of load balancers in AWS?

Type	OSI Layer	Protocol	Key Feature
Classic LB (CLB)	L4 + L7	TCP, HTTP, HTTPS	Legacy — avoid for new projects
Application LB (ALB)	L7	HTTP, HTTPS, WebSocket, gRPC	Host/path routing, WAF, sticky sessions, Lambda targets
Network LB (NLB)	L4	TCP, UDP, TLS	Ultra-low latency, static IPs, source IP preservation
Gateway LB (GWLB)	L3	IP packets	Inline traffic inspection via 3rd-party virtual appliances (firewalls, IDS)

■ *In EKS: NLB fronts the Ingress Controller (NGINX). The AWS Load Balancer Controller can provision ALBs directly from Ingress objects (IngressClass: alb).*

Q13. NLB can also direct external traffic — so why use ALB at all?

NLB works at Layer 4, which means it **cannot inspect HTTP headers, hostnames, or URL paths**. ALB operates at Layer 7 and offers features NLB simply cannot provide:

What ALB can do that NLB cannot:

- **Host-based routing:** Route `api.example.com` to one target group, `app.example.com` to another — on the same ALB.
- **Path-based routing:** `/login` → `auth-service`, `/products` → `catalog-service` — impossible at L4.
- **Header-based routing:** Route based on HTTP headers (e.g., A/B testing, canary releases).
- **AWS WAF integration:** Block malicious requests at the LB level before they reach pods.
- **gRPC and WebSocket support** natively.
- **Lambda functions as targets** — NLB cannot target Lambda.
- **Sticky sessions** via cookies (application-layer concept — NLB can't do this without NLB Proxy Protocol).

When NLB wins:

- Ultra-low latency (microseconds vs milliseconds for ALB).
- Static Elastic IP needed. TLS passthrough. Non-HTTP protocols (MQTT, custom TCP).

■ *Best practice: NLB for the Ingress entry point (static IP, raw TCP) + ALB or NGINX Ingress Controller for L7 routing.*

SECTION 4 — EC2 Access Recovery

Q14. You have an EC2 instance and you've lost the PEM key. How do you connect to it?

Several approaches depending on the situation. The cleanest for modern setups is **AWS Systems Manager Session Manager**.

Method 1 — SSM Session Manager (no PEM needed):

- Requires: SSM Agent installed on EC2, EC2 has IAM role with `AmazonSSMManagedInstanceCore` policy, outbound internet (or VPC endpoint for SSM).
- Connect: AWS Console → Systems Manager → Session Manager → Start Session → select instance.
- Or via CLI: `aws ssm start-session --target i-xxxxxxxxxx`

Method 2 — EC2 Instance Connect (Linux only):

- AWS pushes a temporary public key to the instance for 60 seconds.
- Works only if EC2 Instance Connect is enabled and security group allows port 22 from AWS IP ranges.

Method 3 — Replace the key (stop-based):

- Stop the EC2 instance. Detach the root EBS volume.
- Attach it to another EC2 as a secondary volume.
- Mount it and add your new public key to `/mnt/home/ec2-user/.ssh/authorized_keys`.
- Reattach the volume to the original EC2, start it, connect with new key.

Method 4 — User data script (stop-based):

- Stop instance → Modify User Data → add a script that appends your new public key to `authorized_keys` → start instance.

■ *Best answer: SSM Session Manager. It works without any SSH port open, no PEM, and is fully auditable via CloudTrail.*

Q15. How do you enable Session Manager on an EC2 instance?

Honest interview move: flagging uncertainty and making an educated guess is correct behavior — interviewers respect it over confidently wrong answers.

Requirements to enable SSM Session Manager:

- **SSM Agent:** Pre-installed on Amazon Linux 2/2023, Ubuntu 16.04+, Windows Server 2008+. For older AMIs: install manually.
- **IAM Role:** Attach an IAM Instance Profile with the managed policy `AmazonSSMManagedInstanceCore`.

- **Network access for SSM:** Either outbound HTTPS (port 443) to AWS endpoints (`ssm.region.amazonaws.com`) OR VPC endpoints for SSM (for private subnets with no NAT).

VPC Endpoints needed for fully private Session Manager:

- `com.amazonaws.region.ssm`
- `com.amazonaws.region.ssmmessages`
- `com.amazonaws.region.ec2messages`

Verification:

```
aws ssm describe-instance-information
# Instance should appear in the list
```

■ No inbound security group rule needed — Session Manager uses outbound HTTPS only. Zero open ports.

SECTION 5 — EKS Deployment from GitHub Repository

Q16. A client gives you a GitHub repo and wants it deployed on EKS. What Kubernetes manifests/resources do you create?

Full production deployment requires the following resources, in order of dependency:

1. Namespace

- Logical isolation for the application. Example: `kubectl create namespace my-app`

2. ConfigMap

- Non-sensitive environment configuration: app port, feature flags, external service URLs.

3. Secret

- Sensitive data: DB passwords, API keys. In production: use External Secrets Operator + AWS Secrets Manager. Base64 K8s Secrets are not truly encrypted at rest by default.

4. Deployment

- Defines the pod template, replica count, image, resource requests/limits, liveness and readiness probes, rolling update strategy.

5. Service (ClusterIP)

- Stable internal endpoint for the Deployment. Other services and the Ingress route to this.

6. HorizontalPodAutoscaler

- Auto-scales replicas based on CPU/memory. Requires Metrics Server.

7. Ingress

- Defines HTTP routing rules: `host/path` → Service. Requires an Ingress Controller installed (NGINX or AWS ALB Controller).

8. PodDisruptionBudget (PDB)

- Ensures at least N pods are available during voluntary disruptions (node drains, upgrades).

9. ServiceAccount + RBAC (if needed)

- If the app needs to access AWS resources: ServiceAccount + IAM Role via IRSA (IAM Roles for Service Accounts).

CI/CD pipeline to automate this from GitHub:

- GitHub Actions/Jenkins: Build image → Push to ECR → Update image tag in Helm/Kustomize → `kubectl apply / helm upgrade`.

■ For multi-environment: use Kustomize overlays (base + dev/staging/prod patches) or Helm with values files per environment.

SECTION 6 — Your Questions to the Interviewer

Q17. What was the expectation on Divergent Branch strategy?

This is a sharp question — it shows you're thinking about engineering practices, not just survival.

What they likely meant by Divergent Branch:

- **Long-lived feature branches** that diverge significantly from main — leading to merge conflicts and integration pain.
- SquareOps likely expects a **trunk-based development** or **short-lived branch strategy** to minimize divergence.

Best-practice answer for DevOps context:

- Feature branches should live for **1–3 days max**. Merge to main frequently via PRs.
- Use **feature flags** to deploy incomplete features to production safely without long-lived branches.
- CI pipelines should run on every branch push — catching divergence early.
- In Kubernetes: branch-to-namespace strategy (each branch deploys to its own namespace) allows testing without long divergence.

■ Your ApnaKart project's branch-to-namespace Kustomize strategy directly addresses this. Lead with that in future interviews.

Q18. Which managed Kubernetes vendor is dominating the market and why?

Based on market data and adoption trends, **Amazon EKS** dominates, followed by **GKE** and **AKS**.

Vendor	Market Position	Key Strengths
Amazon EKS	#1 by revenue & enterprise adoption	AWS ecosystem lock-in, IAM/IRSA, Fargate, widest enterprise use
Google GKE	#2, technically strongest	Inventor of Kubernetes, Autopilot mode, best upstream compatibility, GKE Dataplane
Azure AKS	#3, strong in Microsoft-heavy orgs	AAD integration, .NET workloads, hybrid (Arc), free control plane
DigitalOcean DOKS	SMB segment leader	Simplicity, cost — not enterprise grade
CNCF Certified (self-hosted)	On-prem/hybrid	OpenShift (RedHat), Rancher (SUSE) — for on-prem control

- **Why EKS dominates:** AWS has the largest cloud market share (~32%). Most enterprises already run workloads on AWS, making EKS the path of least resistance.
- **Why GKE is technically superior:** Google built Kubernetes. GKE Autopilot, Dataplane v2, and Workload Identity are ahead of EKS equivalents.
- **Why AKS gains ground:** Microsoft's enterprise relationships — if your org uses Azure AD and Office 365, AKS is the natural choice.

■ SquareOps is a multi-cloud managed services provider. Expect them to work across all three. Knowing trade-offs matters more than picking a winner.